

Recap

Managed caches are deployed by service providers to enhance the performance and resilience of their services

CDNs are very sophisticated and expensive proprietary solutions based on content replication across multiple edge servers

These solutions apply to any version of the HTTP protocol

HTTP/2 is the first major revision of HTTP/1.1 that aims at improving web performance by better leveraging network resources

Binary framing is a layer that specifies how exchanges between clients and servers are formatted and transported

HTTP requests and responses are associated with specific frames

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

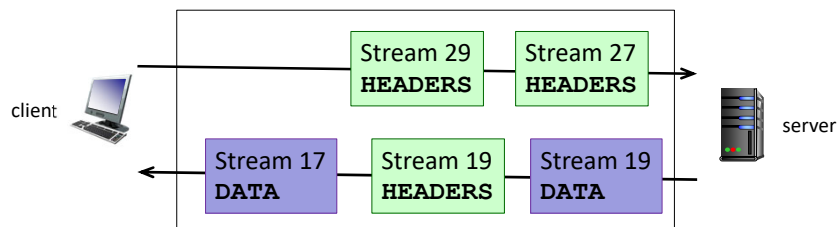
102

Frames & streams

Streams are independent bidirectional series of frames

A client sends an HTTP request on a new stream, using a previously unused stream identifier (identifiers are incremented)

A server sends an HTTP response on the same stream as the request



Streams leverage a single TCP connection

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

103

TCP & HTTP

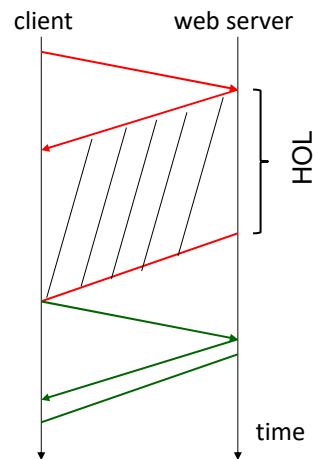
HTTP/2 aims at improving the usage of network resources by overcoming HTTP/1.1 limitations with respect to TCP usage

Under HTTP/1.1 a single TCP connection can be reused for multiple HTTP requests and responses (persistent)

After HTML parsing, the browser knows the URI of each referenced object

It will then send HTTP requests sequentially in a single TCP connection (or in multiple resource-intensive parallel TCP connections)

A “big” response blocks small requests that follow (i.e., Head Of Line blocking)



© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

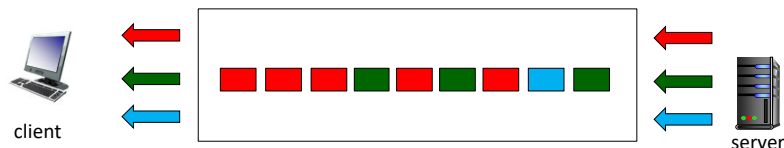
104

TCP & multiplexing

HTTP/2 introduces the concept of multiplexing, that is, HTTP requests and responses can be sent concurrently in a single TCP connection

Multiple requests in parallel without blocking any request and multiple responses in parallel without blocking any response

Multiplexing will reduce the delays and improve the Page Load Time



The order of the frames is flexible (e.g., round robin mechanism)

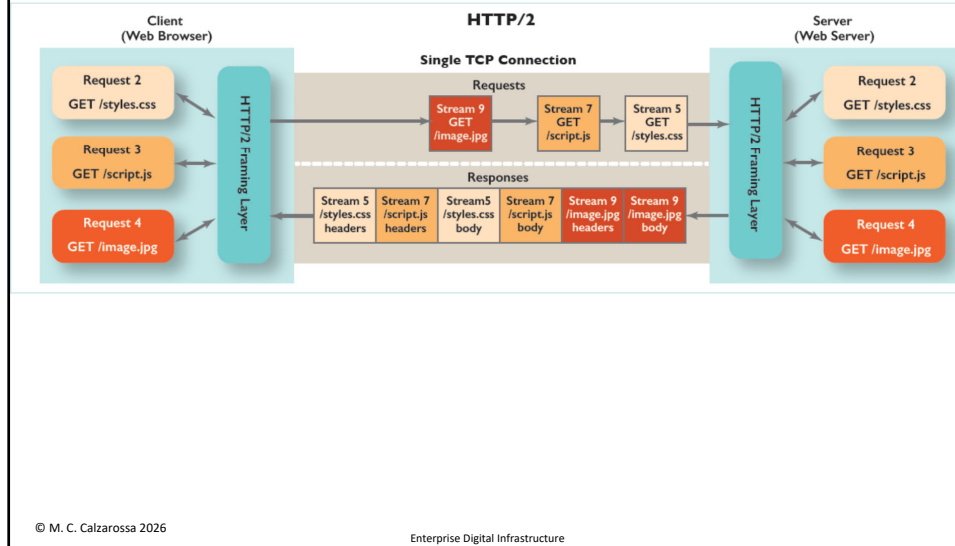
Reassembling performed at the client and at the server side

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

105

More detailed view



106

Flow control

Multiplexing introduces contention over the TCP connection that might result in blocked streams

Effective use of multiplexing depends on flow control and prioritization

Flow control ensures that streams on the same TCP connection do not destructively interfere with each other, i.e., it regulates the rate of data transmission between clients and servers

Flow control is used both at stream level and at connection level

It applies to **DATA** frames only

Implementation is based on the **WINDOW_UPDATE** frame used by clients or servers to advertise how many bytes they are prepared to receive

Initial value of the flow control window set to 64Kbytes

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

107

Prioritization

Multiplexing offers the ability to express how to allocate bandwidth and computation resources across streams by associating priorities with the streams

Prioritization ensures that the limited resources are used most effectively

Priority is used to speedup the delivery of “critical” objects within a page, e.g., style sheets, images

A client can assign a priority for a new stream by including prioritization information in the **HEADERS** frame that opens the stream

At any time, the **PRIORITY** frame can be used to specify the sender-advised priority of a stream

Implementation based on the construction of a prioritization tree

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

108

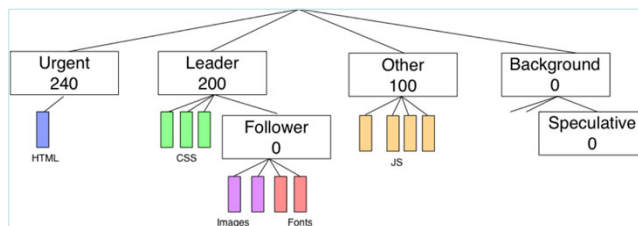
Prioritization tree

Firefox builds a dependency tree that groups resources and schedules the groups to load one after another or to share bandwidth between the groups

Within a given group the resources share bandwidth and download concurrently

Images are scheduled to load after the render-blocking stylesheets and to load in parallel

The render-blocking scripts and stylesheets also load in parallel



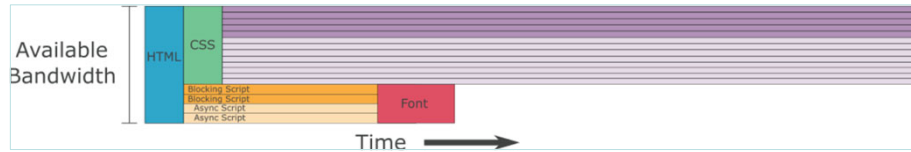
© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

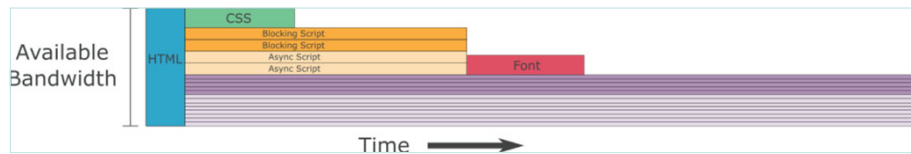
109

Bandwidth usage

Firefox



Safari



[Cloudflare blog on prioritization](#)

HTTP/2 performance is heavily affected by flow control and prioritization settings

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

110

Header fields & compression

HTTP/2 does not change the definitions of HTTP request and response header fields, with only minor exceptions:

1. all header field names are lowercase
2. three *pseudo-header fields* **:method**, **:path**, **:scheme** used to carry control data

```
:authority www.abc.net
:method GET
:path /
:scheme https
```

It might include port number; corresponds to **Host** in HTTP/1.1

control data

Header field compression aims at minimizing the overhead in HTTP request and response messages and the traffic over the network by exploiting redundancy and repetitions

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

111

HTTP request messages

```
GET /lite/static/js/7979.5e7e834d.chunk.js HTTP/2
Host: cdn-client.medium.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:150.0) Gecko/20100101 Firefox/150.0
Accept: */*
Accept-Language: en-US,en;q=0.9
Accept-Encoding: gzip, deflate, br, zstd
Referer: https://medium.com/coderscorner/http-2-flow-control-77e54f7fd518
Connection: keep-alive
Cookie: uid=lo_4b9d2fe3066f; sid=1:EIbLeNwCI5r2k0GVL270twOS3awWYmHUxjk04yPpI7Vvx063ZQeCBiD8q9dtSp5x;
Sec-Fetch-Dest: script
Sec-Fetch-Mode: no-cors
Sec-Fetch-Site: same-site
Pragma: no-cache
```

```
GET /lite/static/js/7975.3f8d607c.chunk.js HTTP/2
Host: cdn-client.medium.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:150.0) Gecko/20100101 Firefox/150.0
Accept: */*
Accept-Language: en-US,en;q=0.9
Accept-Encoding: gzip, deflate, br, zstd
Referer: https://medium.com/coderscorner/http-2-flow-control-77e54f7fd518
Connection: keep-alive
Cookie: uid=lo_4b9d2fe3066f; sid=1:EIbLeNwCI5r2k0GVL270twOS3awWYmHUxjk04yPpI7Vvx063ZQeCBiD8q9dtSp5x;
Sec-Fetch-Dest: script
Sec-Fetch-Mode: no-cors
Sec-Fetch-Site: same-site
Pragma: no-cache
```

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

112

Header compression

Header compression was initially implemented using **zlib** library which reached a good reduction in size (i.e., 85%-88%) but was unsecure (e.g., Compression Ratio Info-leak Made Easy attack leading to session hijacking)

Current implementation relies **HPACK** compression, a method designed for header compression [[RFC 7541 “HPACK: Header Compression for HTTP/2”](#) published in 2015]

HPACK eliminates redundant header fields, limits vulnerability to known security attacks and has bounded memory requirements for constrained environments

Clients and servers maintain and update indexed lists of previously seen header fields (i.e., “history” to establish a shared compression context) to be used as a reference to efficiently encode previously transmitted values

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

113

Implementation

HPACK is based on three main components: two tables and the Huffman encoding scheme

The tables are indexed by (key, value) pairs and associate each header field with an index

1. A [static table](#) consisting of a predefined dictionary of 61 commonly used header fields, some with predefined values. A numeric identifier (from 1 to 61) is associated with each entry
2. An initially empty dynamic table consisting of a list of the actual headers encountered during the decompression. Due to the limited size (`SETTINGS_HEADER_TABLE_SIZE` in `SETTINGS` frame), new entries evict old entries (FIFO approach)

Huffman encoding is based on a static Huffman code computed specifically for HTTP request/response headers. The shortest encoding possible is 5 bits long (highest achievable compression ratio is 8:5 or 37.5% smaller)

The tables are combined into a single address space for defining index values

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

114

HPACK workflow

Workflow of HPACK for encoding a header in the format *name:value* pair

1. It checks the static and dynamic tables. If the full name:value pair is present, it will simply reference the entry in the table [header expressed by 1 or 2 bytes]
2. It checks the static and dynamic tables to find a header with the same name. If the name exists, it will simply reference the entry in the table [name expressed by 1 or 2 bytes] and the value will be encoded using the Huffman encoding
3. If the name is not present, the name:value pair will be encoded

The gains HPACK provides for HTTP request headers are more significant than for response headers

Request headers get better compression, due to much higher duplication

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

115